

PRESENTATION HANDBOOK · 8 SLIDES

Everything you need to say, and everything you might get asked

A full talking script for each slide of Krypssis_Presentation.pptx, plus the questions most likely to come up and prepared answers for each — including the one about the 50% mark.

How to use this

Read once, straight through, the night before. On presentation day, skim the **Script** boxes right before you go up — they're written to be said almost as-is, not just bullet reminders. The **Q&A** boxes are the questions most likely to actually come from an evaluator; don't memorize them word for word, just know the shape of the answer.

- Know which teammate is presenting which slide before you walk in.
- Have the GitHub repo open in a browser tab, in case anyone asks to see real code or commit history.
- Read the “Hard Questions” chapter twice — especially the first one.

0 Title Slide

ON SCREEN

On screen: project title, subtitle, team names, course context.

SCRIPT

“Good [morning/afternoon]. We're presenting Krypsis — a Federated Learning-based Network Intrusion Detection System with a custom communication protocol, for our Computer Networks course project. I'm [name], and with me are [names] — we'll each cover a section.”

What does “Krypsis” mean / why that name?

Krypsis comes from the Greek for “hiding” or “concealment” — fitting, since the whole system is built around never exposing raw data, only sharing what's necessary (model updates) while keeping everything else hidden.

1 Introduction

Why this project, and what it is

ON SCREEN

On screen: why we chose it (privacy problem with centralized NIDS) · what the project is (NIDS + Federated Learning + custom protocol).

SCRIPT

“Every intrusion detection system needs to see a lot of network traffic to be accurate — the more varied the traffic it learns from, the better it gets at spotting real attacks. The problem is, no organization wants to hand its traffic logs to an outside server. That data can reveal internal structure, and in many cases sharing it is against privacy regulation.

Federated Learning solves this by flipping the flow — instead of moving data to one place to train a model, we send the model out to where the data already is. Each client trains locally and only sends back what it learned, never the data itself. That's the foundation of our project. On top of that, we're designing a custom protocol for how those model updates actually get exchanged, instead of just using a generic tool like HTTP.”

Isn't Federated Learning already a solved, well-known idea?

Yes — Federated Learning itself isn't new, and we say that openly. Our contribution isn't inventing FL; it's the custom protocol layer built on top of it, which is where the actual novelty sits (covered in the Roadmap slide).

What exactly is a “client” in your system?

A simulated participant holding its own slice of network traffic data — standing in for a real organization, router, or edge device that would have its own traffic in a real deployment.

2 Application

Where this matters, and what we did

ON SCREEN

On screen: real-world use cases (hospitals/banks, IoT/edge, cross-border) · what we built (5 simulated clients, FedAvg, IID + non-IID tests).

SCRIPT

“This kind of system matters anywhere multiple organizations want to collaborate on security without exposing their internal networks to each other — hospitals sharing threat intelligence, banks, or IoT device networks where each router or gateway does its own local learning, which is exactly the fog computing idea from our syllabus.

What we actually built: five simulated clients, each with their own slice of the NSL-KDD dataset. We trained one shared model across all of them using Federated Averaging. And critically, we tested two scenarios — one where every client's data looks similar, and a harder, more realistic one where clients have very different traffic patterns, the way real organizations actually differ from each other.”

Why does it matter that you tested both IID and non-IID?

Because real organizations never have identical traffic patterns. If a system only works when all clients look the same, it wouldn't survive contact with the real world. Testing the harder, non-IID case proves the approach is realistic, not just theoretical.

3 Tools Used

The toolchain behind it

ON SCREEN

On screen: Python, TensorFlow/Keras, scikit-learn, pandas/NumPy, NSL-KDD dataset, Git/GitHub · note on Mininet/OpenFlow as a future extension.

SCRIPT

“Our tools are a machine-learning implementation stack, not a network simulator — and that's intentional. We used Python as the core language, TensorFlow and Keras to build and train the neural network, scikit-learn for preprocessing and evaluation metrics, and pandas and NumPy for handling the data itself. Our benchmark dataset is NSL-KDD, a standard, widely used network intrusion detection dataset. Everything is version-controlled in Git and hosted on GitHub, so our full development history is visible and inspectable.

We didn't use Mininet or Packet Tracer, because our contribution is at the learning-algorithm and protocol-design level, not network topology — but it's a natural next step: once our custom protocol exists, running it over an actual simulated network in Mininet with OpenFlow would let us test it under real network conditions, which connects directly to the SDN material from Unit 3.”

Why didn't you use a network simulator like Packet Tracer?

Because this project's core contribution is a machine learning + protocol design problem, not a network topology problem. Packet Tracer / Mininet simulate how packets move through routers and switches — useful for testing our custom protocol's real-world network behavior later, but not needed to design and train the detection model itself.

What is NSL-KDD, exactly?

A public benchmark dataset of labeled network connection records — each row describes one network connection (protocol, service, byte counts, flags, etc.) with a label saying whether it was normal traffic or a specific attack. It's the standard academic dataset for testing intrusion detection systems.

4 Methodology

How it was built, step by step

ON SCREEN

On screen: 4-step pipeline (preprocess → split clients → build model → FedAvg loop) · the FedAvg loop explained in 5 bullets.

SCRIPT

“Our pipeline has four stages. First, we preprocess the raw NSL-KDD data — encoding text fields like protocol type into numbers, and scaling every numeric feature to the same range so the neural network can learn from them properly. Second, we split the training data across five simulated clients. Third, we define our model — a Multi-Layer Perceptron, a standard neural network for this kind of tabular data, with layers of 256, 128, and 64 neurons.

Fourth is the actual federated training loop. Each round, the server sends its current shared model to every client. Each client trains it further using only its own data. Then — and this is the key part — clients send back only the updated numbers, the model weights, never the underlying traffic data. The server combines everyone's weights into one improved model, weighted by how much data each client had, and the cycle repeats for fifteen rounds.”

Why weight the averaging by how much data each client has?

Because a client with 40,000 samples has learned from far more evidence than one with 5,000 — giving them equal weight would let the smaller client's noise distort the shared model. This weighted approach is the standard, correct definition of Federated Averaging.

What's a Multi-Layer Perceptron, briefly?

The simplest kind of neural network — layers of neurons fully connected to the next layer, learning to combine input features into a decision. It's the standard choice for tabular, row-of-numbers data like ours, as opposed to something like a CNN, which is built for images.

5 Status & Results

What's done, and the real numbers

ON SCREEN

On screen: 4 result stats (83.3% / 80.2% / 79.1% / 99.0%) · phase completion table, phases 1-6 done, phase 7 in progress.

SCRIPT

“Our centralized baseline — training normally, on all the data at once — reaches 83.3% accuracy. Training the same model federated, with data never leaving each client, reaches 80.2% on evenly split clients and 79.1% on the harder, realistic unevenly split clients. That's within a few points of centralized training, which is the core proof point: we're not paying a heavy accuracy cost for keeping data private.

One more honest number: on an easier evaluation where every attack type is seen during training, the same model reaches 99%. We report both numbers because the harder, official test — which includes attack types the model has never seen — is the more meaningful, realistic measure, and we didn't want to only show the flattering one.”

Why is federated accuracy lower than centralized?

Because each client only trains on its own smaller slice of data before the server averages everyone together — some information gets “smoothed out” in that averaging process. A few points of accuracy loss in exchange for never centralizing raw data is considered a good trade-off in this field.

Why isn't accuracy higher, like 95%+?

Our test set deliberately includes attack types absent from training, specifically to test real generalization rather than memorization — that's a well-known, intentional property of this benchmark. We verified this directly: the same model hits 99% when tested only on attack types it has seen before.

6 Learning

How this connects to our syllabus

ON SCREEN

On screen: three cards mapping the project to Unit 1 (network edge/OSI), Unit 2 (application/transport/network layers), Unit 3 (IoT/fog/SDN).

SCRIPT

“This project isn't separate from what we studied this semester — it's built directly out of it. Every field in our dataset is packet or flow-level data: protocol type maps to the Network layer, connection flags map to the Transport layer, and service maps to the Application layer, straight from Unit 1 and Unit 2.

Designing our custom communication protocol is literally application-layer protocol design — the same exercise as comparing HTTP against FTP against a custom protocol, just applied to model updates instead of web pages. And the Federated Learning architecture itself is a textbook example of fog computing from Unit 3 — local processing at the edge, with only summarized results sent up to a central aggregator, which is the cloud side of that same pattern.”

Can you give one specific example of a dataset feature tied to a specific OSI layer?

Yes — “protocol_type” (TCP/UDP/ICMP) is a Network layer concept, “flag” (like SYN, ACK, connection state) is a Transport layer concept, and “service” (HTTP, FTP, etc.) is an Application layer concept. All three sit in a single row of our dataset.

7 Roadmap

What's left, and exactly how

ON SCREEN

On screen: 3 numbered items — integrity + fingerprint protocol, one poisoning attack, global vs. Mondrian threshold comparison.

SCRIPT

“The first half of our objectives — the working Federated Learning system — is fully complete and proven with real numbers. What's left is the custom protocol itself. Concretely, three things: first, attaching a lightweight integrity tag and a compact statistical fingerprint to every update a client sends, so tampered or suspicious updates can be caught cheaply, before they reach expensive server-side processing.

Second, simulating an actual poisoning attack — a malicious client sending corrupted updates — so we have something real to test detection against. And third, our core experiment: comparing one global suspicion threshold against a threshold calibrated separately per client group, to test whether a single global rule unfairly flags honest, unusual clients as malicious — and whether the smarter, grouped version fixes that without missing real attacks.”

Why present a roadmap instead of just the finished thing?

Because a research project's value isn't only “is it 100% done” — it's “do you understand exactly what's left and why it matters.” We know precisely what the remaining work is, down to the specific statistics involved, which is itself evidence the plan is real, not aspirational.

The hard questions

THE ONE TO ACTUALLY PREPARE FOR

“This looks like less than 50% — is this really a mid-review-ready amount of progress?”

Don't get defensive — lead with what's actually true and strong: “Objective 1 of our five objectives — the entire Federated Learning framework — is 100% complete, tested, and backed by real experimental numbers, not a prototype or a plan. That includes data preprocessing, two different client-heterogeneity scenarios, a tuned model, and a working FedAvg training loop, all documented with honest results, including two ideas we tried and discarded because they didn't work.

What remains is entirely Objective 2 — the custom communication protocol — and what it unlocks in Objectives 4 and 5. That's not a small remaining task, but it's a precisely scoped one: we know exactly what to build, down to the specific statistics the fingerprint check uses and the exact experiment that tests our research question. We'd rather show you a fully working, honestly-evaluated first half than a half-built version of everything.”

If pushed further: “We made a deliberate choice to fully finish and validate the foundation before starting the protocol, rather than build both halves partially and risk neither working. The roadmap slide shows precisely what's next.”

What is the single most novel thing about this project?

Most Federated Learning security research treats “make communication efficient” and “catch malicious clients” as two separate problems, solved at different stages. Our protocol design fuses them — a cheap fingerprint check happens at the transport layer, before the expensive server-side defense runs, and we're specifically testing whether that cheap check treats honest, unusual clients unfairly compared to a smarter, per-group calibrated version.

Why NSL-KDD instead of a newer dataset like CICIDS2017?

NSL-KDD's official test set deliberately includes attack types absent from training — a genuine test of generalization. Most CICIDS2017 usage in the literature uses a random split where every attack type is seen during training, which is an easier, different question. Since our whole research direction is about generalization under difficult, unusual conditions, NSL-KDD's harder evaluation protocol is actually the better fit for this specific project, not a lesser choice.

What would you do differently if you started over?

We'd scope the custom protocol's minimal version earlier, in parallel with finishing the FL baseline, rather than sequentially — though finishing one thing completely before starting the next did keep our results honest and fully verified rather than partially built everywhere.

Rapid-fire glossary

If asked to define something on the spot, these are the one-liners.

NIDS

Software that watches network traffic and classifies it as normal or an attack.

Federated Learning

Training one shared model across many data owners, without any of them sharing their raw data.

FedAvg

The algorithm that combines clients' trained models into one, weighted by how much data each client had.

IID / non-IID

Whether every client's data looks statistically similar (IID) or genuinely different (non-IID, the realistic case).

MLP

Multi-Layer Perceptron — the basic neural network architecture used here, suited to row-of-numbers data.

Dropout

Randomly disabling some neurons during training so the network doesn't over-rely on any one of them.

Overfitting

A model that memorized training data instead of learning patterns that generalize to new data.

Fingerprint (our protocol)

A small set of statistics about a model update, used to cheaply flag suspicious ones before expensive processing.

Mondrian calibration

Using a separate detection threshold per data subgroup instead of one global threshold — our planned fix for unfair flagging.